

Remove Duplicates From an Unsorted Linked List

My Personal DSA Notes — POTD

1. Problem Statement

Given the head of an unsorted linked list, find all values that appear more than once and delete **every node** having any of those values. Return the remaining linked list.

Example 1 — input

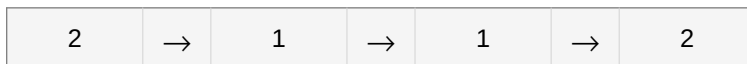


Example 1 — output

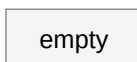


Since 2 occurs more than once, **both** nodes containing 2 are deleted.

Example 2 — input



Example 2 — output



Both 2 and 1 occur more than once, so every node is deleted.

2. Key Observation

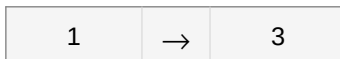
The important part is that we are not simply removing the second occurrence. If a value is duplicated, **all occurrences of that value must be removed**.

For example:

All 2s must disappear



Result



3. Approach 1 — HashMap Frequency

First count how many times every value occurs. Then traverse the linked list again. Keep a node only when its frequency is exactly 1. If its frequency is greater than 1, skip that node.

The frequency expression:

```
freq.put(temp.val, freq.getDefault(temp.val, 0) + 1);
```

In a few words: get the current count (or 0 if absent), add 1, and store the new count back in the HashMap.

4. Complete HashMap Solution

```
class Solution {  
    public ListNode deleteDuplicatesUnsorted(ListNode head) {
```

```

    if (head == null || head.next == null) {
        return head;
    }

    HashMap<Integer, Integer> freq = new HashMap<>();

    // Count the frequency of every value.
    ListNode temp = head;

    while (temp != null) {
        freq.put(temp.val, freq.getOrDefault(temp.val, 0) + 1);
        temp = temp.next;
    }

    // Dummy node makes deleting the head easy.
    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode prev = dummy;
    temp = head;

    while (temp != null) {
        if (freq.get(temp.val) > 1) {
            // Delete the current node.
            prev.next = temp.next;
        } else {
            // Keep the current node.
            prev = temp;
        }

        temp = temp.next;
    }

    return dummy.next;
}

```

5. Why Do We Need the Dummy Node?

Suppose the first node itself is duplicated:

Original



If we try to delete the head directly, special handling is required. A dummy node gives us a node before the head:

With dummy



Then deletion always follows the same rule:

```
prev.next = temp.next;
```

Finally, **dummy.next** is the head of the remaining list.

6. Step-by-Step Example

Input: 1 → 2 → 3 → 2

Frequency map:

1 → 1
2 → 2
3 → 1

Second traversal:

- 1 has frequency 1 → keep it.
- 2 has frequency 2 → delete it.
- 3 has frequency 1 → keep it.
- Last 2 has frequency 2 → delete it.

Final result



7. Approach 2 — No HashMap (O(1) Extra Space)

We can avoid the HashMap by repeatedly counting how many times the current value appears in the entire linked list. If the count is greater than 1, remove every occurrence of that value.

This removes the O(n) HashMap, but it requires repeated traversals, so the time complexity becomes O(n²).

8. Complete O(1)-Space Solution

```
class Solution {  
  
    public ListNode deleteDuplicatesUnsorted(ListNode head) {  
  
        if (head == null || head.next == null) {  
            return head;  
        }  
  
        ListNode dummy = new ListNode(0);  
        dummy.next = head;  
  
        ListNode current = head;  
  
        while (current != null) {  
  
            // Count how many times current value appears.  
            int count = 0;  
            ListNode temp = head;  
  
            while (temp != null) {  
                if (temp.val == current.val) {  
                    count++;  
                }  
                temp = temp.next;  
            }  
  
            // If duplicated, remove every occurrence.  
            if (count > 1) {  
  
                ListNode prev = dummy;  
                temp = head;  
  
                while (temp != null) {  
  
                    if (temp.val == current.val) {  
                        prev.next = temp.next;  
                    } else {  
                        prev = temp;  
                    }  
  
                    temp = temp.next;  
                }  
            }  
  
            current = current.next;  
        }  
    }  
}
```

```

        current = current.next;
    }

    return dummy.next;
}

```

9. Important Correction to a Common Mistake

A tempting $O(1)$ -space idea is to check only whether the current value appears *later* in the list. That is not sufficient by itself because the final occurrence of a duplicated value has no later duplicate.

Example:

The last 2 has no later 2



The algorithm must remember that 2 was duplicated earlier, or count the value across the whole list. The $O(1)$ -space solution above handles this correctly.

10. Complexity Comparison

Approach	Time	Extra Space
HashMap frequency	$O(n)$	$O(n)$
No HashMap	$O(n^2)$	$O(1)$

Time-optimal choice: HashMap approach — $O(n)$ time. No algorithm can do better than $O(n)$ time because every node must be examined to determine which values are duplicated.

11. Which One Should You Remember?

For interviews, remember the **HashMap frequency approach** first. It gives $O(n)$ time and is straightforward.

If the interviewer specifically says **$O(1)$ extra space**, use the no-HashMap approach and accept the $O(n^2)$ time trade-off.

12. Edge Cases

- **Empty list:** return null.
- **One node:** return the node because it cannot be duplicated.
- **All values unique:** return the original list unchanged.
- **All values duplicated:** return null.
- **Duplicate at the head:** dummy node handles it cleanly.
- **Consecutive duplicates:** every occurrence must be removed.
- **Non-consecutive duplicates:** every occurrence must also be removed.

13. Quick Revision

Goal: Remove all nodes whose value occurs more than once.

Best time approach: HashMap frequency counting.

Pass 1: count frequencies.

Pass 2: delete nodes where frequency > 1 .

Dummy: simplifies head deletion.

Time: $O(n)$.

Space: $O(n)$.

$O(1)$ -space alternative: $O(n^2)$ time.

14. Final Intuition

Think of the problem as **count first, filter second**. The HashMap tells you whether a value is safe to keep. A value with frequency 1 stays; a value with frequency greater than 1 is removed everywhere.